

Improvement Heuristic For Solving The One-Dimensional Bin-Packing Problem

Sofiene Abidi

F.S.J.E.G.J.,

University of Jendouba

Tunisia, Jendouba

Email: abidisofiene@hotmail.fr

Saoussen Krichen

F.S.J.E.G.J.,

University of Jendouba

Tunisia, Jendouba

Email: saoussen.krichen@isg.rnu.tn

Enrique Alba

E.T.S.I. Informática,

Universidad de Málaga

Spain, Málaga

Email: eat@lcc.uma.es

Juan M. Molina

E.T.S.I. Informática,

Universidad de Málaga

Spain, Málaga

Email: jmmb@lcc.uma.es

Abstract—We develop in the present paper a genetic algorithm for the one-dimensional bin packing problem. This algorithm performs a series of perturbations in an attempt to improve the current solution, applying some problem dependant genetic operators. Our procedure is efficient and easy to implement. We apply it to several benchmark instances taken from some problem sets and compare our results to those found in the literature. We find that our algorithm is able to generate competitive results compared to the best methods known so far and computes, for the first time, one optimal solution for one open benchmark instance.

I. INTRODUCTION

The one-dimensional bin packing problem (BPP) is defined as follows: given a set of n elements of various weights $w_1, w_2, w_3, \dots, w_n$, is it possible to pack all items into N bins each one with capacity C ? The main objective of the BPP is to minimize the number of used bins without exceeding the capacity C for each bin. Equivalently, it can be viewed partitioning a set of n positive numbers into a minimum number of subsets such that the sum of the numbers in each subset is lesser than or equal to a fixed value. Even though this problem is simple to state, it is $\mathcal{NP}$ hard, it is unlikely that there exists an algorithm that could solve the BPP in a polynomial time.

The BPP has a large influence on the industrial sector, for example in scheduling applications (planning advertising spots, scheduling tasks on processors, scheduling tasks with resource constraints) and in transport field (loading trucks with weight constraints or with volume constraint, filling containers, pallets).

In general, restricting all the feasible solutions to find the best solution, seems to be impossible even for a problem of medium size. Therefore, we can address the resolution of the BPP by means of approximate methods, especially heuristics and meta-heuristics methods. As with many other meta-heuristics used to solve these problems, the success of genetic algorithms (GAs) is, to a large extent, due to its ability to recover the search process from getting stuck in a local optimum. GAs has shown a remarkable efficiency on many problems (scheduling, packing and vehicle routing) and often surpass the best solutions previously found by other approaches.

Various solutions approaches were proposed for the BPP. For many years it was and still is the subject of many researches, Martello and Toth (1990) solved the BPP using a branch-

and-bound procedure MTP (Martello and Toth procedure), this is the basic reference used in most comparative studies. A hybrid grouping genetic algorithm was presented by Falkenauer (1996). Coffman et al. (1997) provided a complete study of approximation methods for this problem. Scholl et al. (1997) proposed an exact method (BISON), combining tabu search with a branch-and-bound procedure, using a new branching scheme, this paper also contains several bounds for the problem. Schwerin and Wäscher (1997) based on the cutting stock problem, developed a new lower bound for the BPP and embedded it in MTP, which give him high-quality results with their procedure (MTPCS). Fleszar and Hindi (2002) describes an improved implementation of Gupta and Ho (1999) heuristic minimum bin slack (MBS), and designed a modified version called MBS', as well as a variable-neighborhood-search meta-heuristic. Alvim et al. (2004) solved the BPP with an hybrid improvement heuristic (HI-BP) that uses tabu search to move items between bins. Fleszar and Charalambous (2011) introduced the new sufficient average weight (SAW) principle, used to improve both solutions and processing times of a bin-oriented heuristics by considering multiple subsets of items for each bin. This new improvement heuristic outperforms all other known complex heuristics, in terms of both average solution quality and computation time. Loh et al. (2008) proposes a simple and fast heuristic based on the concept of weight annealing (WA) to solve the BPP. Singh and Gupta (2007) proposed two heuristics for the one dimensional BPP, one is a hybrid steady-state genetic algorithm, whereas the other is an improved version of MBS' heuristic.

In this paper, we develop a new procedure based on the genetic algorithm approach to solve the one-dimensional bin packing problem (GA-BPP). The GA-BPP is an improvement heuristic, that performs a series of perturbations in an attempt to improve the initial solution by, applying customized genetic operators. Experimental tests generate competitive results comparable with the best methods known so far, and for one instance we could find a solution better than the best known. We have applied our algorithm to 1210 instances from Scholl et al. (1997) benchmark problem sets. GA-BPP realized the existing best or optimal solutions to 930 instances. Also, GA-BPP could solve an instance of set Gau-1 (Wäscher and Gau (1996)) to optimality for the first time and for all other instances in Gau-1, solutions using the same number of bins as the known best solutions were found.

The paper is organized as follows, in section 1 we have

presented an introduction to the one dimensional BPP and the main researches. In Section 2, we outline and present the main ingredients of the GA-BPP. Results of comprehensive computational experiments are reported in Section 3. Finally, conclusions and further research directions are discussed in Section 4.

II. SOLUTION APPROACH

Genetic algorithms [gold89] are search techniques based on natural selection. A GA updates a population iteratively of solutions, for obtaining better solutions. In a GA new solutions (offsprings) are generated from the incumbent solutions (parents) by applying crossover and mutation operations, crossover operations should be designed in such a way that offsprings inherit important factors of parents.

A. Individual Encoding

The encoding of a chromosome is designed in a tabular form that expresses the items numbers as boxes indices to be assigned to bin numbers. For example, in fig1 item 1 is loaded in bin 3.

Items	1	2	3	4	5	6
Bin	3	1	2	1	4	2

Fig. 1. Encoding schema

B. Initial Population

The algorithm starts with an initial population of 100 chromosomes generated using both the greedy algorithm, the First Fit heuristics (FF) and randomly.

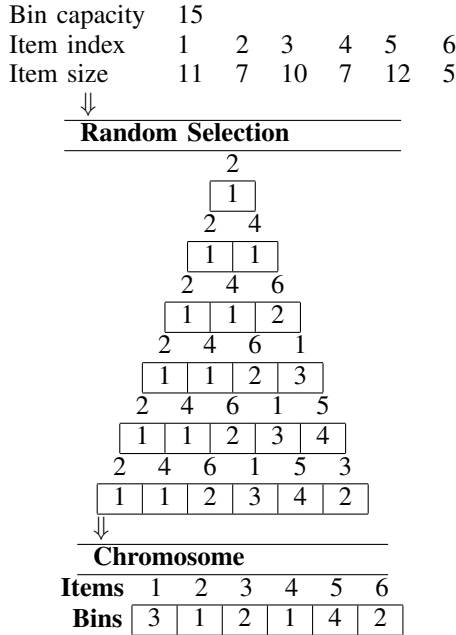


Fig. 2. An illustrative schema for initial solution using FF

In the example reported in fig2 a new chromosome is obtained 312142 using the FF.

The FF heuristic, iteratively picks an object at random from the list of unassigned objects and places it into the first possible bin without violating the capacity constraint. If no such bin exists, a new bin is used. This process is repeated until the list of unassigned objects becomes empty. For the random generation, the bins are loaded sequentially until saturate. This operation is repeated until all the items are assigned.

C. The Fitness Function

Each solution needs to be awarded a figure of merit, to indicate how close it comes to meet the overall specification. This is achieved by applying a fitness function. Previous studies have found that using the number of bins as a fitness measure is not effective as the resulting fitness landscape contains numerous local optima that use one more than the optimal number of bins. Faulkenauer et al. (1996) proposed the following fitness function which is generally used to assess solutions g for the one dimensional BPP:

$$\text{Fitness}(g) = \frac{1}{N} \sum_{i=1}^N \left(\frac{F_i}{C}\right)^2$$

where N is the number of bins, C the maximum capacity of a bin, and F_i is the fullness of the bin. The fullness of a bin i is the sum of the sizes of the items placed in this bin $F_i = \sum_{j=1}^n \delta_{ij} W_j$ where n is the number of items, W_j the weight of the item j and δ_{ij} is 1 if item j is packed in bin i , and 0 otherwise. We shall use this evaluation function to calculate the fitness of the chromosomes. With this function the fitter individual has the higher fitness value.

D. Crossover and Mutation

To enhance the solutions selected in the previous level, they are included into an improvement process using the crossover and mutation operators. However, in our case the number of bins is not fixed and the crossover attempts simultaneously, to reduce the number of bins while preserving the feasibility of the solutions. The tournament selection method is used to select parents to apply the crossover. First individuals are randomly chosen from the population to form the current tournament. Hence, the first two fittest elements of the tournament become the parents of the next generation. The most appropriate size of the tournament is problem dependant, for the GA-BPP the size of the tournament is 10. The crossover operation aims, in some way, at the offsprings inherit important factors of the parents. We have adopted the single point crossover: one crossover point P is selected at random depending on the number of items. One offspring is generated by inheriting the bin assignment of one parent of items from the beginning to the crossover point. The rest of the items are assigned by maintaining the bin assignment of the other parent while it's possible. Else the FF heuristic assigns the left unassigned item. Let's consider, for example the problem of packing six items 1, 2, 3, 4, 5, 6 with respective weights 6, 6, 5, 4, 5, 4, into bins. Each bin has a capacity 10. Let's consider the following two chromosomes be $P1 : 132442$, $P2 : 123344$. Applying the crossover operation to these chromosomes outputs the following offsprings: Off1: 132321 and Off2: 123132.

The first part of P1 is reported in Off1, next we try to copy the other part from P2 to Off1 but considering item by item.

Starting from the first item in the second part of P2, we see if the bin of that item exists in Off1 then we check the slack in the bin if it is larger than the weight of the item, we maintain it in this bin, else if the bin of the item exists in Off1 but can't handle the item we check if there's a bin in Off1 which can handle that item, else the item is packed into a new bin. This process is repeated until all items from the second part are packed.

Off2 is generated by exchanging the role of P1 and P2 in the previous explanation.

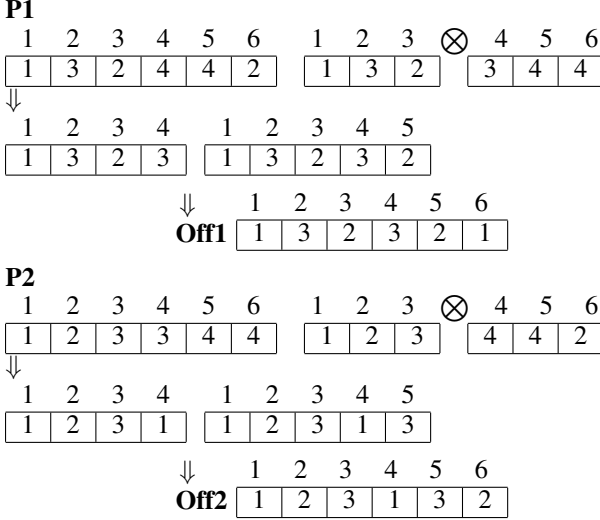


Fig. 3. An illustrative schema for the crossover operation

The next step corresponds to the mutation operation. The offset having the minimum number of bins is selected. If both having the same number of bins we choose randomly one of them. We consider two different swapping schemes proposed by Fleszar and Hindi (2002): swap (1,0), swap (1,1), in an attempt to disrupt the current solution.

In swap (1,0), the bin having the lowest fullness is selected and all its items are moved in other bins, if possible.

In swap (1,1), items are exchanged between all possible pairs of bins. The swapping of item i from bin X and item j from bin Y , is made if the problem constraints are respected.

After mutation, we use the elitism in order to generate the next population. Elitism can very rapidly increase performance of the GA, because it prevents losing the best found solution. We select the best result from the generated offsets and add it to the initial population in place of the solution that have the worst fitness.

The process of moving from the current population to the next population constitutes one generation in the execution of the GA. This process ends when a number of iteration is reached.

E. GA-BPP Algorithm

The GA-BPP algorithm starts with an initial population of size 100. Generated using both the FF procedure to enhance the solutions quality of the initial population and the randomly, opening a single bin then assigning randomly a set of items to it, then closing the bin if there's no more space. To evaluate each solution we calculate the fitness value for all individuals in the population using the fitness function

explained previously. The tournament selection method is used to select two parents to apply genetic operators to. 10 individuals are randomly selected from the population to form the current tournament, then the first two elements of the tournament those with the highest fitness value, are selected as the parents of the next generation, as a first step towards the improvement of the selected solutions. We include them into an improvement process using the crossover operation and the mutation operation. The crossover operation aims, in some way, at the offsprings inherit the assigning block of bins from the parents. The mutation operation seeks to liberate the most empty bin by the means of swap (1,0) and disrupt the current solution by using swap (1,1). The obtained solution from the improvement process, will be incorporated into the initial population instead of the solution that have the lowest fitness value. The GA-BPP algorithm ends when it reaches 1000 iteration.

Algorithm 1 The GA-BPP Algorithm

- 1: Generate the initial population of 100 individuals 50% using FF and 50% randomly generated
- 2: Calculate the fitness values of all individuals in the population
- 3: **repeat**
- 4: Select 10 solutions randomly from the population as the candidates for a tournament.
- 5: Select the two best solutions P1 and P2 from the tournament, using the fitness function, as the parents
- 6: Generate two offsprings s3 and s4 by means of the crossover operation.
- 7: Pick up the best solution between s3 or s4
- 8: Apply the mutation swap(1,0) and swap(1,1) operation to generate s5.
- 9: Remove the worst solution in the population and replace it by s5 if it's better.
- 10: **until** the number of iteration is reached.

III. COMPUTATIONAL STUDY ON BENCHMARKS

Our algorithm is coded in C using Dev-C++ in Win32 mode. All tests were performed on a computer with an Intel Core 2 Duo CPU at 1.6 GHz, with 2GB of RAM. In all our experiments, we have used a population size of 100 individuals. We run our GA until a maximum number of iterations depending on the problem configuration.

In order to examine and evaluate the performance of our GA-BPP, we tested it on two benchmark problem instances. The first was developed by Wäscher and Gau (1996), called gau-1, and contains 17 problem instances, and the second benchmark problem instances, developed by Scholl et al. (1997), contains 1210 problem instances.

As most of the previous works on BPP, we report the time for the overall processing in seconds rather than the time to find the best solution.

Parameter	Description	Value
T	Population size	100
S	Tournament selection size	10
K	Number of iteration	1000

TABLE I. PARAMETER SETTINGS OF GA-BPP

A. Gau Benchmark Instances

The first problem instances Gau-1, contain a hard collection according to their authors composed of 17 problem instances, collected from the paper of Wäscher and Gau (1996). For 14 of these instances, optimal solutions were previously known. The weights are chosen as integer values from the given interval[2, 7332] using uniformly distributed random numbers, with $C = 10,000$ and various values for the number n of items [56, 239]. This problem set is available online at <http://paginas.fe.up.pt/esicup/index.php>.

The fact that they are very old problem instances and still opened until now gave an important motivation to investigated it, in addition they are included in many works and no one rise to solve all the problem instances to optimality. The item sizes may affect the algorithms performance: wide range, very large and very small items, with repetitions.

The results obtained by GA-BPP are reported in Table II. It is noticeable that GA-BPP is able to solve all instances of set Gau-1 and compute a solution better than the best-known for one of the three opened instances: for TEST0022, a solution using 14 instead of 15 bins was found. For all other instances in Gau-1, solutions using the same number of bins as previously best-known were found.

ID	Instances	LB	Best	GA-BPP	Avg time(s)
1*	TEST0022	14	15	14	0.15
2	TEST0065	15	16	16	1.17
3	TEST0097	12	12	12	0.65
4	TEST0058	20	20	20	3.97
5	TEST0055	15	15	15	0.88
6	TEST0049	11	11	11	0.96
7	TEST0075	13	13	13	2.59
8	TEST0054	14	14	14	0.94
9	TEST0068	12	12	12	1.32
10	TEST0014	23	24	24	3.99
11	TEST0082	24	24	24	0.34
12	TEST0044	14	14	14	1.18
13	TEST0030	27	27	27	0.54
14	TEST0005	28	28	28	0.59
15	TEST0095	16	16	16	1.03
16	TEST0055	20	20	20	2.41
17	TEST0084	16	16	16	0.40

TABLE II. GA-BPP RESULTS FOR 17 INSTANCES OF SET GAU-1

*: New best-known solution found in this paper by GA-BPP.

Our computational experiments for the Gau benchmarks showed that GA-BPP performed better than the others works with all respect, and able to solve one instances of set Gau-1 to optimality for the first time (results taken from the literature). Like in the previous works, we performed a robustness test by running the GA-BPP fifty times, Perturbation-SAWMBS (2011), Alvim et al. (2004) and Singh and Gupta (2007) running just five times (numbers taken from respective papers). TEST0022: Solved with number of iteration rise up to 1000, and found the optimal solution overall the runs.

B. Scholl Benchmark Instances

The second benchmark problem instances, developed by Scholl et al. (1997) up to three data sets contain 1210 problem instances, for 1184 of which an optimal solution was found by Scholl, for the remaining 26 instances, optimal solutions were found by Alvim et al. (2004).

The first data set (720 instances) is constructed similar to that used by Martello and Toth (1990) for testing their procedure MTP, the weights are chosen as integer values from the given intervals [1,100], [20,100], and [30,100] using uniformly distributed random numbers, with capacities $C = 100, 120, 150$, and numbers of items $n = 50, 100, 200, 500$.

The second data set (480 instances) contains parameter settings not covered by the first set, with $C = 1000$, $n = 50, 100, 200, 500$ and item weights are given such that the average number of items per bin is three, while the third set contains 10 hard instances with settings ($C = 100,000$, $n = 200$) guarantees that the weights are widely spread and the number of items per bin lies between 3 and 5.

Comparison GA-BPP for this benchmark with related works in the literature, concludes that GA-BPP gives results that can be comparable with the best methods known so far, such as, BISON, an exact method combining tabu search with a branch-and-bound procedure, MTPCS (1997) with a new lower bound for the BPP embedded in MTP, MBS' (2002), an improved implementation of MBS, with a variable-neighborhood-search meta-heuristic, HI-BP (2004), a hybrid improvement heuristic that uses tabu search to move items between bins, PMBS' + VNS hybrid algorithm that combines MBS' and variable neighborhood search, and uses the lower bounds of Fekete and Schepers and WABP(2008), a simple and fast heuristic based on the concept of weight annealing (WA).

The GA-BPP can't outperforms the other methods from the viewpoint of the number of optimal solutions, in contrast, in terms of computation time except the MBS'. Due to the fact that GA-BPP is an unsophisticated procedure and easy to implement.

The next table resume, the results provided by the GA-BPP and the related works depict the number of optimal solutions and the average time for the solutions.

Algorithm	Data set1		Data set2		Data set3		Total
	Hits	Avg time(s)	Hits	Avg time(s)	Hits	Avg time(s)	
GA-BPP	615	0.13	315	0.019	00	0.359	930
MBS'	629	00	381	0.06	00	0.07	1051
BISON	697	32.4	473	16.3	3	700.2	1173
HI-BPP	720	0.19	480	0.01	10	4.60	1210
PMBS'+VNS	694	0.15	474	0.10	02	3.74	1170
MTPCS	717	7.3	394	221	5	3164.8	1116
WABP	720	0.17	480	0.19	10	0.13	1210

TABLE III. COMPARISON OF GA-BPP HITS FOR SCHOLL BENCHMARK

We can interpret that the performance of the GA-BPP decreases from the first data set to the third one, in terms of number of hits down from one set to another, due to the average deviation increases also. In fact, that because of the degree of hardness of the instances which increase from one level to the next.

The GA-BPP scored 76% of optimal solutions for Scholl benchmark instances. For Gau benchmark instances GA-BPP outputs 100% of the best known solutions. Indeed, for the first instance (Test0022) we improved the solution from 15 bins to 14 bins. The GA-BPP performs better to Gau benchmark instances.

RECAP

Several important remarks are noteworthy:

- The GA-BPP improvement heuristic outperforms the best of the known improvement heuristics in terms of solution quality for Gau benchmark instances in a regular frame time.
- Furthermore, a new solution better than the best known was found for one benchmark instance.
- The GA-BPP is found to be the best known or optimal solutions to 930 instances for Scoll benchmark instances and outperforms the other methods in terms of computation time.

IV. CONCLUSION

We developed a new procedure noted GA-BPP based on the genetic algorithm technique, to solve the one-dimensional BPP problem.

GA-BPP is easy to understand, easy to follow, and easy to implement. It generated competitive quality solutions in a regular frame time.

When applied to 1210 benchmark instances, GA-BPP is found to be the best known or optimal solutions to 930 instances and able to solve all the instances of set Gau-1 and is found as the solution better than the best-known for one of the three opened instances.

In further work, we hope to investigate other variants of the bin packing problem and try to apply our procedure for two or more dimensional bin-packing problem.

ACKNOWLEDGMENTS

This work has been funded by the Ministry of Higher Education and Scientific Research in Tunisia, in cooperation with the University of Jendouba, Tunisia : Faculty of Law, Economics and Management <http://www.fsjegj.rnu.tn>.

Third and fourth authors acknowledge funds from the Spanish government and European FEDER under contract TIN2011-28194 (roadME) <http://roadme.lcc.uma.es>.

We are so grateful for their support to us.

REFERENCES

- [1] Alvim, A.C.F., Ribeiro, C.C., Glover, F., Aloise, D.J., 2004. *A hybrid improvement heuristic for the one-dimensional bin packing problem.*, Journal of Heuristics 10 (2), 205-229.
- [2] Carolina.S, et al, 2008. *Greedy Seeding Procedure for GAs Solving a Strip Packing Problem*, Inteligencia Artificial 40, 73-85.
- [3] Coffman.E.G, Garey.M.R and Johnson.D.S, 1997. *Approximation algorithms np-hard problems,chapter 2,pages 46-93.*, D.Hochbaum(ed),PWS Publishing, Boston.
- [4] Dyckhoff.H et Finke.S, 1992. *Cutting and packing in production and distribution, typology and bibliography.* (eds. Mueller, W.A. and Schuster, P.), Springer-Verlag., New York :39-56.
- [5] Fleszar.K, Charalambous.C, 2011. *Average-weight-controlled bin-oriented heuristics for the one-dimensional bin-packing problem*, European Journal of Operational Research 210, 176-184.
- [6] Falkenauer, E., 1996. *A hybrid grouping genetic algorithm for bin packing*, Journal of Heuristics 2, 5-30.
- [7] Fleszar.K, Hindi.K.S, 2002. *New heuristics for one-dimensional bin-packing*, Computers and Operations Research 29: 821-839
- [8] Goldberg. D.E., 1989. *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley.
- [9] Gupta, J.N.D., Ho, J.C., 1999. *A new heuristic algorithm for the one-dimensional binpacking problem.*, Production Planning and Control 10 (6), 598-603.
- [10] Hitosbi.I, Tetsuya.Y, 2003. *A New Design of Genetic Algorithm for Bin Packing*, IEEE: 1044-1049.
- [11] Loh.K.H. et al., 2008. *Solving the one-dimensional bin packing problem with a weight annealing heuristic*, Computers & Operations Research 35, 2283-2291.
- [12] Martello.S, Toth.P. 1990. *Knapsack Problems*. Wiley, available online on <http://www.or.deis.unibo.it/knapsack.html>.
- [13] Schwerin, P., Wascher, G., 1997. *The bin-packing problem: A problem generator and some numerical experiments with FFD packing and MTP.*, International Transactions in Operational Research 4 (5/6), 377-389.
- [14] Scholl.A, Klein.R, Jurgens.C, 1997. *A fast hybrid procedure for exactly solving the one-dimensional bin packing problem.*, Computers and Operations Research 24 (7), 627-645.
- [15] Singh.A, Gupta.A.K, 2007. *Two heuristics for the one-dimensional bin-packing problem.*, OR Spectrum 29:765-781
- [16] Schwerin.P, Wascher.G, 1997. *The bin-packing problem: A problem generator and some numerical experiments with FFD packing and MTP.*, International Transactions in Operational Research 4 (5/6), 377-389.
- [17] Wascher.G, Gau.T, 1996. *Heuristics for the integer one-dimensional cutting stock problem: A computational study.*, OR Spectrum 18 (3), 131-144.